

Various Categories of Memory Technology

Computer Architecture -

SKU 2.3.1

Introduction to Memory Technology

- ▶ • Memory stores instructions and data for processing.
- ▶ • Speed, cost, and volatility determine memory type.
- ▶ • Organized in a hierarchy for performance efficiency.

Memory Hierarchy Concept

- ▶ • Registers
- ▶ • Cache Memory (L1, L2, L3)
- ▶ • Main Memory (RAM)
- ▶ • Secondary Storage (SSD/HDD)
- ▶ • Tertiary Storage

Volatile vs Non-Volatile Memory

- ▶ • Volatile: Requires power (e.g., RAM).
- ▶ • Non-volatile: Retains data without power (e.g., ROM, Flash, SSD).

Primary Memory (Main Memory)

- ▶ • Includes DRAM and SRAM.
- ▶ • Directly accessed by CPU.
- ▶ • Determines system performance significantly.

DRAM (Dynamic RAM)

- ▶ • Stores data as charge in capacitors.
- ▶ • Needs periodic refresh.
- ▶ • Higher density, lower cost, slower than SRAM.

SRAM (Static RAM)

- ▶ • Uses flip-flops for storage.
- ▶ • Fast and expensive.
- ▶ • Used mainly in cache memory.

Cache Memory

- ▶ • Small, fast memory close to CPU.
- ▶ • Levels: L1 (fastest), L2, L3.
- ▶ • Reduces average memory access time.

ROM (Read-Only Memory)

- ▶ • Permanent storage.
- ▶ • Used for firmware.
- ▶ • Types: PROM, EPROM, EEPROM.

Flash Memory

- ▶ • Non-volatile solid-state memory.
- ▶ • Used in USB drives, SSDs.
- ▶ • Faster and more durable than HDDs.

Secondary Memory (Magnetic & Optical)

- ▶ • Magnetic: HDDs store data on spinning disks.
- ▶ • Optical: CDs/DVDs use laser-based storage.
- ▶ • Slower but higher capacity.

SSD vs HDD Technology

- ▶ • SSD: Flash-based, fast, no moving parts.
- ▶ • HDD: Mechanical, cheaper per GB.
- ▶ • SSD preferred for performance-critical systems.

Emerging Memory Technologies

- ▶ • MRAM - Magnetic RAM.
- ▶ • PRAM - Phase Change Memory.
- ▶ • 3D XPoint - High-speed non-volatile memory.
- ▶ • Used in next-generation computing.

Memory Delay Affects Runtime

Computer Architecture -
SKU 2.3.2

Introduction to Memory Delay

- ▶ • Memory delay refers to the time required to access data.
- ▶ • CPU runtime is highly dependent on memory access speed.
- ▶ • Major contributor to system performance bottlenecks.

Sources of Memory Delay

- ▶ • DRAM latency
- ▶ • Cache misses
- ▶ • Bus congestion
- ▶ • Slow I/O devices
- ▶ • Virtual memory page faults

CPU-Memory Performance Gap

- ▶ • CPU cycles are extremely fast compared to memory access.
- ▶ • Known as the memory wall.
- ▶ • Requires architectural techniques to minimize delays.

Cache Memory and Delay Reduction

- ▶ • Caches store frequently accessed data.
- ▶ • Cache hits significantly reduce delay.
- ▶ • Cache misses cause costly main memory access.

Types of Cache Misses

- ▶ • Compulsory miss
- ▶ • Capacity miss
- ▶ • Conflict miss
- ▶ • Each type contributes differently to runtime overhead.

Memory Access Time Formula

- ▶ • Average Memory Access Time (AMAT):
- ▶ $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
- ▶ • Lower miss rate and faster memory reduce runtime.

Pipeline Stalls Due to Memory Delays

- ▶ • Memory delays cause pipeline bubbles.
- ▶ • Stalls reduce instruction throughput.
- ▶ • Modern CPUs use multiple techniques to hide latency.

Techniques to Reduce Memory Delay

- ▶ • Prefetching
- ▶ • Increasing cache size
- ▶ • Multi-level caches
- ▶ • Memory interleaving
- ▶ • Faster memory technologies

Virtual Memory and Runtime Delay

- ▶ • Page faults cause extremely long delays.
- ▶ • Disk access is millions of times slower than RAM.
- ▶ • Efficient memory management reduces page faults.

Impact on Overall Execution Time

- ▶ • Memory delay affects CPI (Cycles Per Instruction).
- ▶ • High CPI = slower program runtime.
- ▶ • Optimizing memory access improves performance drastically.

Memory Management Principles

Computer Architecture -
SKU 2.3.3

Introduction to Memory Management

- ▶ • Memory management controls how memory is allocated and used.
- ▶ • Ensures efficient, safe, and fast data access.
- ▶ • Key to overall system performance and stability.

Objectives of Memory Management

- ▶ • Maximize CPU utilization
- ▶ • Provide isolation between processes
- ▶ • Efficient memory allocation
- ▶ • Protect memory from unauthorized access

Logical vs Physical Address Space

- ▶ • Logical address: generated by CPU.
- ▶ • Physical address: actual location in RAM.
- ▶ • Memory Management Unit (MMU) converts logical to physical addresses.

Contiguous Memory Allocation

- ▶ • Simple method: each process gets a single contiguous block.
- ▶ • Issues: fragmentation, limited flexibility.
- ▶ • Requires compaction to reduce fragmentation.

Fragmentation Types

- ▶ • Internal fragmentation: unused space inside allocated memory.
- ▶ • External fragmentation: scattered free memory blocks.
- ▶ • Compaction helps reduce external fragmentation.

Paging

- ▶ • Memory divided into fixed-size blocks: pages and frames.
- ▶ • Eliminates external fragmentation.
- ▶ • Requires page table for address translation.

Segmentation

- ▶ • Divides memory into variable-sized segments.
- ▶ • Useful for modular programming.
- ▶ • Suffers from external fragmentation.

Paging vs Segmentation

- ▶ • Paging: fixed size, eliminates fragmentation.
- ▶ • Segmentation: variable size, supports logical groupings.
- ▶ • Many systems combine both (segmented paging).

Virtual Memory Principles

- ▶ • Allows execution of programs larger than physical memory.
- ▶ • Uses disk space as an extension of RAM.
- ▶ • Supports demand paging for efficiency.

Page Faults and Performance

- ▶ • Occur when required page is not in RAM.
- ▶ • Causes long delays due to disk access.
- ▶ • Minimizing page faults improves performance.

Memory Protection

- ▶ • Prevents processes from accessing others' memory.
- ▶ • Techniques: base and limit registers, page permission bits.
- ▶ • Ensures system security and stability.

Modern Memory Management Techniques

- ▶ • Multi-level paging
- ▶ • TLB (Translation Lookaside Buffer)
- ▶ • Inverted page tables
- ▶ • Memory-mapped files
- ▶ • Cache-aware memory allocation

Functionality of a System That Uses Virtual Memory Management

Computer Architecture -
SKU 2.3.4

Introduction to Virtual Memory

- ▶ • Virtual memory extends physical memory using disk space.
- ▶ • Allows execution of programs larger than RAM.
- ▶ • Provides isolation and efficient memory usage.

Why Virtual Memory is Needed

- ▶ • Physical memory is limited.
- ▶ • Reduces need for programmers to manage memory manually.
- ▶ • Protects processes from one another.

Virtual Address Space

- ▶ • CPU generates virtual addresses.
- ▶ • MMU translates virtual to physical addresses.
- ▶ • Enables abstraction of large continuous memory.

Paging in Virtual Memory

- ▶ • Memory is divided into pages and frames.
- ▶ • Only required pages are loaded into RAM.
- ▶ • Reduces fragmentation and improves efficiency.

Demand Paging

- ▶ • Pages loaded only when needed.
- ▶ • Minimizes memory usage.
- ▶ • Causes page faults if page not found in RAM.

Page Fault Handling

- ▶ • Generated when required page is not available in memory.
- ▶ • OS loads the page from disk to RAM.
- ▶ • CPU resumes execution after page is loaded.

Page Replacement Algorithms

- ▶ • Used when RAM is full.
- ▶ • Algorithms include FIFO, LRU, Optimal.
- ▶ • Determines which page to evict for new pages.

Translation Lookaside Buffer (TLB)

- ▶ • Cache for address translations.
- ▶ • Stores recent virtual-to-physical mappings.
- ▶ • Reduces address translation time significantly.

Advantages of Virtual Memory

- ▶ • Enables multitasking.
- ▶ • Provides memory protection.
- ▶ • Efficient use of RAM.
- ▶ • Supports large applications.

Performance Considerations

- ▶ • Page faults cause slowdowns.
- ▶ • TLB hit ratio affects performance.
- ▶ • Disk speed impacts virtual memory efficiency.

Virtual Memory in Modern Systems

- ▶ • Widely used in Windows, Linux, macOS.
- ▶ • Supports advanced features like memory-mapped files.
- ▶ • Essential for modern multi-process operating systems.

Memory Access Time Under Different Cache and Memory Configurations

Computer Architecture -
SKU 2.3.5

Introduction to Memory Access Time

- ▶ • Memory access time is the delay to read/write data.
- ▶ • Strongly influenced by cache levels and memory technology.
- ▶ • A key factor in system performance and CPU efficiency.

Hierarchy and Access Time

- ▶ • Registers: Fastest, accessed in <1 ns.
- ▶ • Cache (L1, L2, L3): Small but fast.
- ▶ • Main Memory (DRAM): Slower, larger.
- ▶ • Secondary Storage: Very slow compared to RAM.

Average Memory Access Time (AMAT)

- ▶ • AMAT formula:
- ▶ $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
- ▶ • Lower miss rate and faster memory improve performance.

Cache Hit and Miss

- ▶ • Hit: Data found in cache → fast access.
- ▶ • Miss: Data fetched from lower-level memory → slow.
- ▶ • Types: compulsory, capacity, conflict misses.

L1, L2, L3 Cache Comparison

- ▶ • L1: Smallest, fastest (1-2 cycles).
- ▶ • L2: Larger, slower (5-20 cycles).
- ▶ • L3: Shared, largest, slower (20-40 cycles).
- ▶ • Each level reduces average memory delay.

Impact of Cache Size

- ▶ • Larger cache → fewer misses.
- ▶ • But larger cache may have longer hit time.
- ▶ • Designers balance size and speed.

Impact of Cache Associativity

- ▶ • Direct-mapped: Fastest but highest conflict misses.
- ▶ • Set-associative: Balanced performance.
- ▶ • Fully associative: Lowest misses but slowest lookup.

Memory Latency in DRAM

- ▶ • DRAM latency depends on row access and refresh cycles.
- ▶ • Access times typically 50-100 ns.
- ▶ • Affects overall memory performance significantly.

Effect of Multi-Core Processors

- ▶ • Shared caches create contention.
- ▶ • More cache coherence traffic increases latency.
- ▶ • Requires optimized cache-coherence protocols.

Techniques to Reduce Access Time

- ▶ • Multi-level caches
- ▶ • Prefetching
- ▶ • Memory interleaving
- ▶ • Faster DRAM (e.g., DDR5)
- ▶ • Improved cache replacement algorithms

Data Transfers and I/O Control Implemented via Interrupts

Computer Architecture -
SKU 2.3.6

Introduction to Interrupt-Driven I/O

- ▶ • Interrupts allow devices to notify the CPU when attention is needed.
- ▶ • Eliminates constant polling by the CPU.
- ▶ • Improves efficiency during data transfers.

What Are Interrupts?

- ▶ • Signals generated by hardware or software.
- ▶ • Temporarily halts CPU execution.
- ▶ • CPU executes Interrupt Service Routine (ISR).

Types of Interrupts

- ▶ • Hardware interrupts: From I/O devices.
- ▶ • Software interrupts: System calls.
- ▶ • Maskable and non-maskable interrupts.

Interrupt-Driven I/O Process

- ▶ 1. Device generates interrupt.
- ▶ 2. CPU pauses current task.
- ▶ 3. Executes ISR to handle data.
- ▶ 4. Resumes interrupted task.

Polling vs Interrupts

- ▶ • Polling: CPU repeatedly checks device status.
- ▶ • Interrupts: Device alerts CPU when ready.
- ▶ • Interrupts reduce CPU overhead.

Direct Memory Access (DMA)

- ▶ • Allows devices to transfer data to RAM without CPU involvement.
- ▶ • Significantly reduces CPU workload.
- ▶ • Works alongside interrupt systems.

DMA and Interrupts Relationship

- ▶ • DMA signals CPU when transfer completes.
- ▶ • CPU uses interrupt to finalize the operation.
- ▶ • Common in disk, network, and audio devices.

Interrupt Priority Levels

- ▶ • Ensures urgent tasks are handled first.
- ▶ • High-priority devices interrupt low-priority ones.
- ▶ • Critical for real-time systems.

Interrupt Vector Table

- ▶ • Table of ISR addresses.
- ▶ • CPU uses interrupt number to locate correct ISR.
- ▶ • Enables fast and organized interrupt handling.

Advantages of Interrupt-Driven I/O

- ▶ • Efficient CPU utilization.
- ▶ • Faster response to I/O events.
- ▶ • Better multitasking and system throughput.

Applications of Interrupt-Based I/O

- ▶ • Keyboard and mouse input.
- ▶ • Disk read/write operations.
- ▶ • Network communication.
- ▶ • Timer-based scheduling.

Types of Buses in a Computer System

Computer Architecture -
SKU 2.3.7

Introduction to Computer Buses

- ▶ • A bus is a communication pathway for data transfer.
- ▶ • Connects CPU, memory, and I/O devices.
- ▶ • Types: Data, Address, and Control buses.

Data Bus

- ▶ • Transfers actual data between components.
- ▶ • Bidirectional.
- ▶ • Width (8, 16, 32, 64 bits) affects system performance.

Address Bus

- ▶ • Carries memory addresses from CPU to RAM or I/O.
- ▶ • Unidirectional.
- ▶ • Width determines maximum addressable memory.

Control Bus

- ▶ • Sends control signals (read/write, interrupt, clock).
- ▶ • Coordinates activities among components.
- ▶ • Examples: MEMR, MEMW, IOR, IOW signals.

System Bus

- ▶ • Combination of data, address, and control buses.
- ▶ • Connects CPU and main memory.
- ▶ • Basis of system performance.

Expansion Bus

- ▶ • Connects peripheral devices to motherboard.
- ▶ • Examples: PCI, PCIe, USB.
- ▶ • Supports device expansion and upgrades.

Front-Side Bus (FSB)

- ▶ • Connects CPU to main memory controller.
- ▶ • Used in older architectures.
- ▶ • Replaced by modern high-speed interconnects.

Back-Side Bus (BSB)

- ▶ • Connects CPU to L2 cache.
- ▶ • Faster than FSB.
- ▶ • Often dedicated for cache communication.

PCI and PCI Express

- ▶ • PCI: Parallel, slower.
- ▶ • PCIe: Serial, high-speed lanes.
- ▶ • Widely used for GPUs, SSDs, and network cards.

USB Bus

- ▶ • Universal Serial Bus for external devices.
- ▶ • Hot-swappable.
- ▶ • Versions: USB 2.0, 3.0, 3.1, 3.2, USB4.

Serial vs Parallel Buses

- ▶ • Parallel: multiple bits at once (e.g., old PCI, IDE).
- ▶ • Serial: one bit at a time, higher frequency (PCIe, SATA).
- ▶ • Serial designs improve speed and reduce noise.

Modern High-Speed Buses

- ▶ • SATA (storage)
- ▶ • Thunderbolt (high-speed external I/O)
- ▶ • NVMe (PCIe-based storage)
- ▶ • Offer low latency and high throughput.

Using a Magnetic Disk Drive for Data Access

Computer Architecture -
SKU 2.3.8

Introduction to Magnetic Disk Drives

- ▶ • Magnetic disk drives (HDDs) use rotating platters.
- ▶ • Data stored magnetically on disk surfaces.
- ▶ • Common secondary storage in computer systems.

Disk Structure Overview

- ▶ • Consists of platters, tracks, sectors, and cylinders.
- ▶ • Read/write head accesses data.
- ▶ • Controlled by disk controller hardware.

Platters, Tracks, and Sectors

- ▶ • Platter: circular disk storing data.
- ▶ • Track: concentric circle on a platter.
- ▶ • Sector: smallest addressable unit on disk.

Cylinders

- ▶ • Set of tracks vertically aligned across platters.
- ▶ • Enables simultaneous access to aligned tracks.
- ▶ • Reduces seek operations for sequential access.

Disk Access Mechanisms

- ▶ • Three main components:
 - ▶ 1. Seek time
 - ▶ 2. Rotational latency
 - ▶ 3. Transfer time
- ▶ • Total access time = sum of all above.

Seek Time

- ▶ • Time to move read/write head to target track.
- ▶ • Usually 2-10 ms.
- ▶ • Major contributor to disk delay.

Rotational Latency

- ▶ • Time waiting for desired sector to rotate under head.
- ▶ • Depends on RPM (e.g., 7200 RPM $\approx$ 4.2 ms avg latency).

Transfer Time

- ▶ • Time to read/write data once positioned.
- ▶ • Affected by disk speed and interface bandwidth.

Disk Scheduling Algorithms

- ▶ • Improve efficiency by minimizing head movement.
- ▶ • Common algorithms:
 - ▶ • FCFS
 - ▶ • SSTF
 - ▶ • SCAN
 - ▶ • C-SCAN

Sequential vs Random Access

- ▶ • Sequential access is much faster.
- ▶ • Random access causes frequent seeks.
- ▶ • File system layout impacts performance.

Caching in Disk Drives

- ▶ • HDDs use onboard cache (8-128 MB).
- ▶ • Reduces access time for frequently used data.
- ▶ • Works similar to memory caching principles.

Magnetic Disk Advantages

- ▶ • Low cost per GB.
- ▶ • High storage capacity.
- ▶ • Good for archival or bulk storage.

Limitations of Magnetic Disk Drives

- ▶ • Slow compared to SSDs.
- ▶ • Mechanical wear and tear.
- ▶ • Sensitive to physical shock.

Use Cases of HDD Storage

- ▶ • Data centers requiring large cold storage.
- ▶ • Backup and archival systems.
- ▶ • Desktop PCs and servers with mixed storage tiers.

Famous Network Structures: Ethernet & Bus

Computer Architecture -
SKU 2.3.9

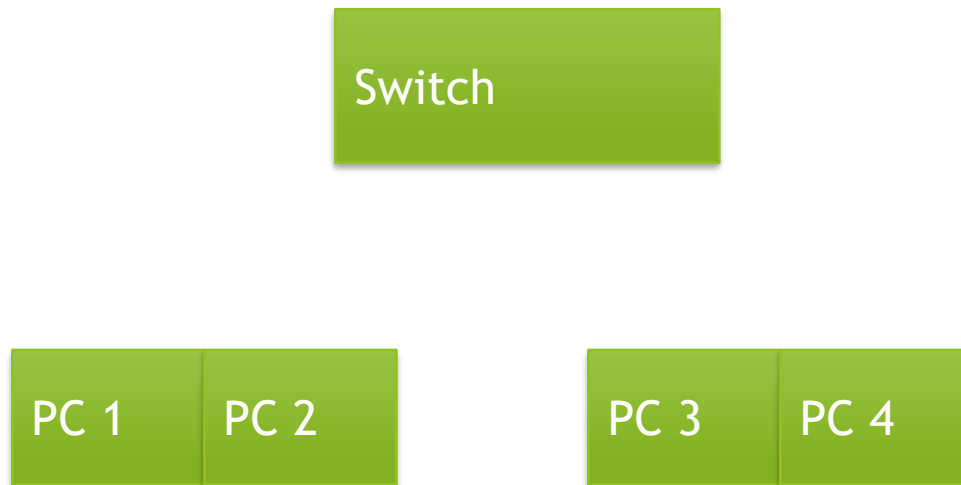
Introduction to Network Structures

- ▶ • Network structures define how devices are interconnected.
- ▶ • Influence reliability, speed, scalability, and cost.
- ▶ • Common structures: Ethernet, Bus, Ring, Star, Mesh.

What is Ethernet?

- ▶ • A widely used LAN communication protocol.
- ▶ • Uses CSMA/CD (older versions) for medium access.
- ▶ • Supports high-speed data transfer (1Gbps-400Gbps).
- ▶ • Uses switches in modern networks rather than shared media.

Ethernet Switch-Based Architecture (Diagram)



Bus Network Architecture

- ▶ • All devices share a single communication line.
- ▶ • Used in early LANs and computer backplanes.
- ▶ • Simple but limited scalability.
- ▶ • Collision handling required (e.g., CSMA/CD).

Bus Network (Diagram)



Bus Line

Comparison

- ▶ Ethernet:
 - ▶ • Modern switched networks
 - ▶ • High performance & scalable
 - ▶ • Reduced collisions
- ▶ Bus:
 - ▶ • Simple and cost-effective
 - ▶ • Limited bandwidth
 - ▶ • Suitable for small systems or backplane designs

Applications

- ▶ • Ethernet: LANs, data centers, enterprise networks.
- ▶ • Bus structures: embedded systems, system buses (PCI, ISA).

Summary

- ▶ • Ethernet is the dominant modern LAN technology.
- ▶ • Bus networks are simpler but limited.
- ▶ • Both play important roles in communication architecture.

The Basics of RAID Architectures

Course: Computer Architecture -
SKU 2.3.10

What is the Problem? The Pre-RAID Era

- ▶ Single Large Expensive Disk (SLED):
- ▶ Performance was limited by the single disk's speed.
- ▶ A single disk failure meant total data loss and significant downtime.
- ▶ Capacity was limited to the largest available disk.
- ▶ The "Reliability vs. Performance" Trade-off:
- ▶ How can we get both faster data access and better data protection without a massive cost increase?
- ▶ Key Goal: Use multiple, smaller, cheaper disks to act as a single, large, reliable, and high-performance logical unit.
- ▶ Image: An icon of a single, large, old-fashioned hard drive with a "fragile" or "warning" symbol on it.
- ▶ Notes:
"Before RAID, systems often relied on a Single Large Expensive Disk, or SLED. This created a bottleneck both in terms of performance and reliability. If that one disk failed, everything was lost. The core idea behind RAID is to use an array of smaller, cheaper, industry-standard disks to overcome these limitations."

What is RAID?

- ▶ RAID = Redundant Array of Independent Disks.
- ▶ A data storage virtualization technology that combines multiple physical disk drive components into one or more logical units.
- ▶ Key Concepts:
- ▶ Data Striping: Splitting data across multiple disks.
- ▶ Data Mirroring: Copying data to multiple disks.
- ▶ Parity: A calculated value used to reconstruct data in case of a drive failure.
- ▶ It is a classic example of interfacing and managing multiple storage devices at the system level.
- ▶ Notes:
"So, RAID is a method of interfacing multiple independent disks to present them as a single logical unit to the operating system. Its magic lies in three key techniques: Striping for performance, Mirroring for redundancy, and Parity for a balance of both."

Key Concept 1: Data Striping

- ▶ Data is split into "blocks" and spread across multiple disks in the array.
- ▶ Analogous to: A team of workers each taking a section of a long document to type simultaneously.
- ▶ Benefits:
- ▶ Performance: Multiple disks can be read from or written to concurrently.
- ▶ High Throughput for large, sequential data accesses.
- ▶ Drawback:
- ▶ No Redundancy. The failure of any single disk results in complete data loss.
- ▶ Notes:
"Striping is all about performance. By splitting a file across several disks, we can read or write all parts of it simultaneously. This is like parallel processing for your storage. However, it's important to note that striping alone provides NO redundancy. It actually increases the risk of failure because if any one disk in the array fails, the entire file is corrupted."

Key Concept 2: Data Mirroring

- ▶ Data is duplicated identically on two or more disks.
- ▶ Analogous to: Keeping a real-time, identical backup copy.
- ▶ Benefits:
 - ▶ High Redundancy: Can tolerate the failure of one (or more, depending on setup) disks.
 - ▶ Read Performance: Can read from either copy, potentially doubling read speed.
- ▶ Drawback:
 - ▶ High Cost: 50% (or more) storage overhead. For 1TB of data, you need 2TB of physical space.
- ▶ Notes:

"Mirroring is the simplest form of redundancy. Every piece of data is written to two separate disks. This means you can lose one disk and keep running without any data loss. It can also improve read performance. The major downside is the cost; you're effectively paying double for your storage capacity."

Key Concept 3: Parity

- ▶ Parity is a checksum or an XOR-based calculation of the data stored across the other disks.
- ▶ If a drive fails, the missing data can be mathematically recalculated from the remaining data and the parity information.
- ▶ Benefit:
- ▶ Provides redundancy with a much lower storage overhead than mirroring (e.g., only 1 disk's worth of capacity for the entire array).
- ▶ Drawback:
- ▶ Write Penalty: Calculating and writing parity requires extra I/O operations, which can slow down writes.
- ▶ Notes:
"Parity is a clever space-saving redundancy technique. Instead of keeping a full copy of the data, we store a special calculated value. If a disk fails, we can use this parity data, along with the data on the good disks, to reconstruct what was on the failed drive. The trade-off is a performance hit on write operations."

Common RAID Levels - An Overview

- ▶ RAID levels define how data and redundancy are distributed.
- ▶ RAID 0: Striping only. (Performance)
- ▶ RAID 1: Mirroring only. (Redundancy)
- ▶ RAID 5: Block-level striping with distributed parity. (Good Balance)
- ▶ RAID 6: Block-level striping with double distributed parity. (High Redundancy)
- ▶ RAID 10 (1+0): A stripe of mirrors. (High Performance & Redundancy)
- ▶ Notes:
"Now let's look at how these techniques are combined into standardized 'RAID Levels.' Each level offers a different mix of performance, capacity, and fault tolerance."

RAID 0: Striping

- ▶ Technique: Data striping only.
- ▶ Minimum Disks: 2
- ▶ Advantages:
 - ▶ Excellent performance (both read and write).
 - ▶ Full storage capacity utilization (zero overhead).
- ▶ Disadvantages:
 - ▶ No fault tolerance. A single drive failure results in total data loss.
- ▶ Use Case: Non-critical data requiring high speed (e.g., video editing scratch disks, gaming).
- ▶ Notes:

"RAID 0 is pure striping. It gives you the best performance and all the raw capacity, but it's like walking a tightrope without a net. One disk fails, and your entire array is gone. Use it only for temporary or easily replaceable data where speed is the only concern."

RAID 1: Mirroring

- ▶ Technique: Data mirroring only.
- ▶ Minimum Disks: 2
- ▶ Advantages:
 - ▶ Excellent redundancy (can survive loss of one disk).
 - ▶ Good read performance.
- ▶ Disadvantages:
 - ▶ High cost (50% storage overhead).
 - ▶ Write performance is similar to a single disk.
- ▶ Use Case: Small databases, critical system drives.
- ▶ Notes:

"RAID 1 is the opposite: pure mirroring. It's simple and very safe. You get good read performance and can survive a disk failure. The major cost is that you only get to use half of the total physical storage capacity. It's great for small, critical volumes."

RAID 5: Striping with Distributed Parity

- ▶ Technique: Block-level striping with parity distributed across all disks.
- ▶ Minimum Disks: 3
- ▶ Advantages:
 - ▶ Good read performance.
 - ▶ Fault tolerance for a single drive failure.
 - ▶ Better storage efficiency than RAID 1 (Overhead = $1/n$, e.g., 33% for 3 disks).
- ▶ Disadvantages:
 - ▶ Write performance penalty due to parity calculation.
 - ▶ Slow rebuild process on large disks.
- ▶ Use Case: General purpose file and application servers.
- ▶ Notes:

"RAID 5 is a very popular compromise. It uses striping for performance and adds single-disk fault tolerance through distributed parity. The parity is not on a dedicated disk but spread across all disks, which prevents a bottleneck. The overhead is just one disk's worth of capacity for the whole array. The main downside is the 'write penalty'."

RAID 6: Striping with Double Parity

- ▶ Technique: Block-level striping with two independent parity schemes.
- ▶ Minimum Disks: 4
- ▶ Advantages:
 - ▶ Can survive the failure of any two disks.
 - ▶ Essential for large arrays where a second drive might fail during a long rebuild process.
- ▶ Disadvantages:
 - ▶ Higher storage overhead than RAID 5 ($2/n$, e.g., 50% for 4 disks).
 - ▶ Even greater write performance penalty than RAID 5.
- ▶ Use Case: Large-scale storage, archival systems, where data integrity is paramount.
- ▶ Notes:

"RAID 6 takes the concept of RAID 5 a step further by using two sets of parity. This means it can withstand the simultaneous failure of any two disks in the array. This is crucial for very large arrays where the rebuild time after one failure is long, and the risk of a second failure during that time is unacceptably high."

Nested RAID: RAID 10 (1+0)

- ▶ Technique: A stripe (RAID 0) of mirrored pairs (RAID 1).
- ▶ Minimum Disks: 4
- ▶ Advantages:
 - ▶ Excellent performance (both read and write).
 - ▶ High redundancy (can survive multiple drive failures, as long as they are not in the same mirror pair).
- ▶ Disadvantages:
 - ▶ High cost (50% storage overhead, same as RAID 1).
- ▶ Use Case: High-performance databases, I/O-intensive application servers.
- ▶ Notes:

"RAID 10, often called 'one-plus-zero', combines the best of both worlds. First, you mirror disks for safety, then you stripe across those mirrored pairs for speed. You get fantastic performance and excellent redundancy. The cost is still 50% overhead, but for many high-demand applications, it's the preferred solution."

Comparison Summary

- ▶ (Survives failure of one drive in each mirror pair)*
- ▶ Notes:
"Your choice of RAID level is always a trade-off. There is no single 'best' level; it depends entirely on your requirements for performance, capacity, and data protection."

The RAID Controller (Interfacing)

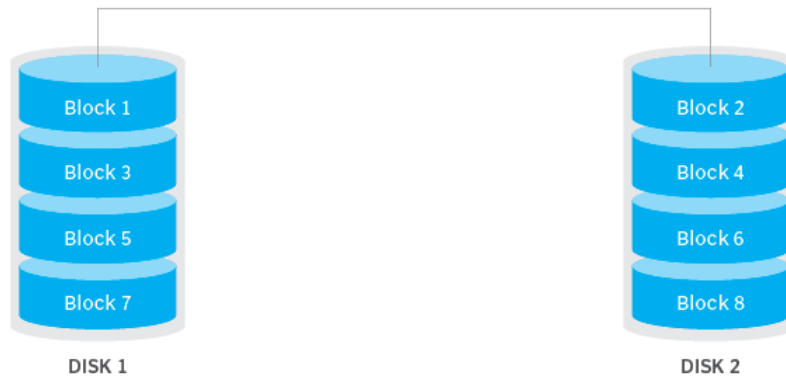
- ▶ The "brain" of the RAID array.
- ▶ Hardware Controller: A dedicated PCIe card with its own processor, memory (cache), and sometimes a battery backup unit (BBU).
- ▶ Offloads parity calculations from the main CPU.
- ▶ Provides better performance.
- ▶ Software RAID: The operating system handles the RAID management.
- ▶ Cheaper, but uses host system CPU and memory.
- ▶ (e.g., Windows Storage Spaces, Linux mdadm).
- ▶ Manages the abstraction of the physical disks into the logical volume presented to the OS.
- ▶ Notes:
"From an interfacing perspective, the key component is the RAID Controller. This can be a dedicated hardware card that handles all the complex striping, mirroring, and parity calculations, presenting a simple, single drive to the operating system. Software RAID is a cheaper alternative but uses your main system resources."

Slide 15: Summary & Key Takeaways

- ▶ Content:
- ▶ RAID is a fundamental storage technology for performance, capacity, and reliability.
- ▶ It is built on three core techniques: Striping, Mirroring, and Parity.
- ▶ Different RAID Levels (0,1,5,6,10) offer different trade-offs.
- ▶ The choice depends on application needs: Cost vs. Performance vs. Data Protection.
- ▶ RAID is not a backup! It protects against hardware failure, not against data corruption, accidental deletion, or malware.
- ▶ Notes:
"In conclusion, RAID is a critical tool in a system architect's toolkit for building robust storage subsystems. Remember the key trade-offs, and most importantly, remember that RAID is about uptime and availability, not about backup. You still need a separate backup strategy to protect your data from all threats."

RAID 0

Striping



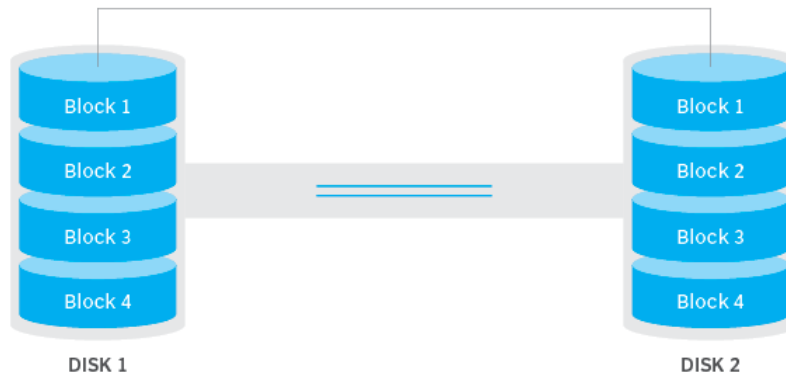
©2018 TECHTARGET. A

RAID 0

RAID 0 uses striping to improve data access performance but doesn't provide data redundancy.

RAID 1

Mirroring

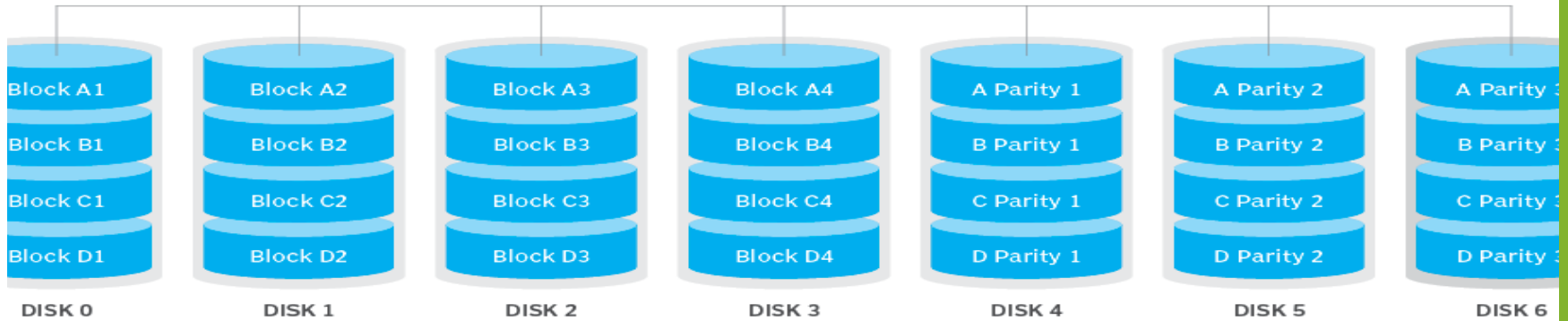


©2018 TECHTARGET. A

RAID 1

RAID 1 mirrors all data writes to at least two different devices providing 100% data redundancy.

RAID 2



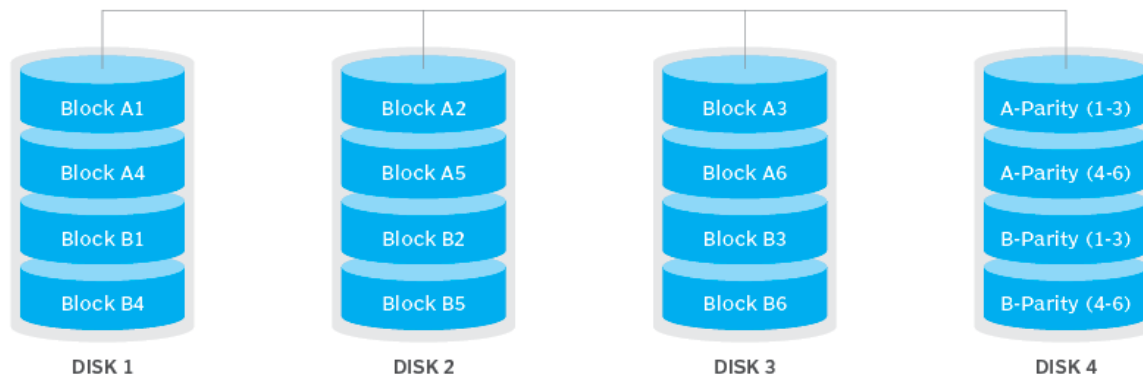
© 2018 TECHTAR

RAID 2

RAID 2 stripes data across devices and incorporates error correction technology.

RAID 3

Parity on separate disk

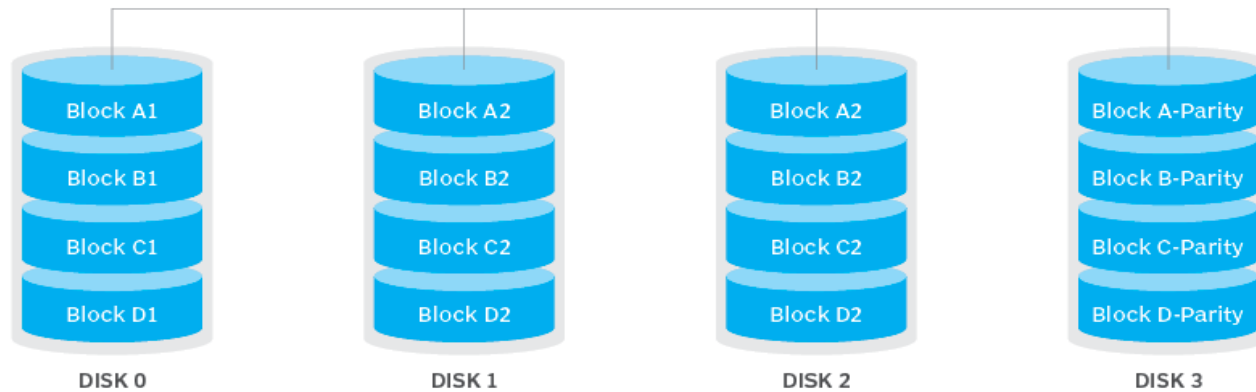


©2018 TECHTARGET. A

RAID 3

RAID 3 stripes data across drives and uses a drive to store parity information to enable rapid recoveries.

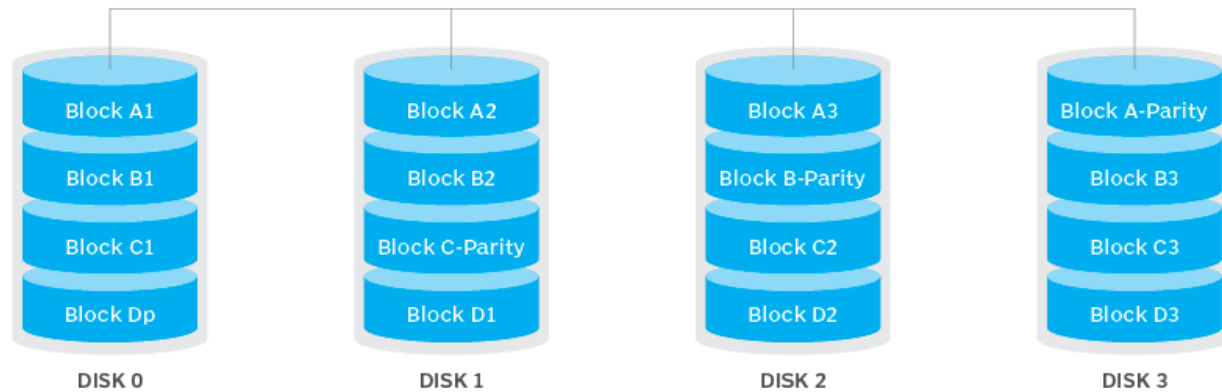
RAID 4



RAID 4

RAID 4 is similar to RAID 3, but it uses larger data stripes to accommodate different types of data and access requirements.

RAID 5

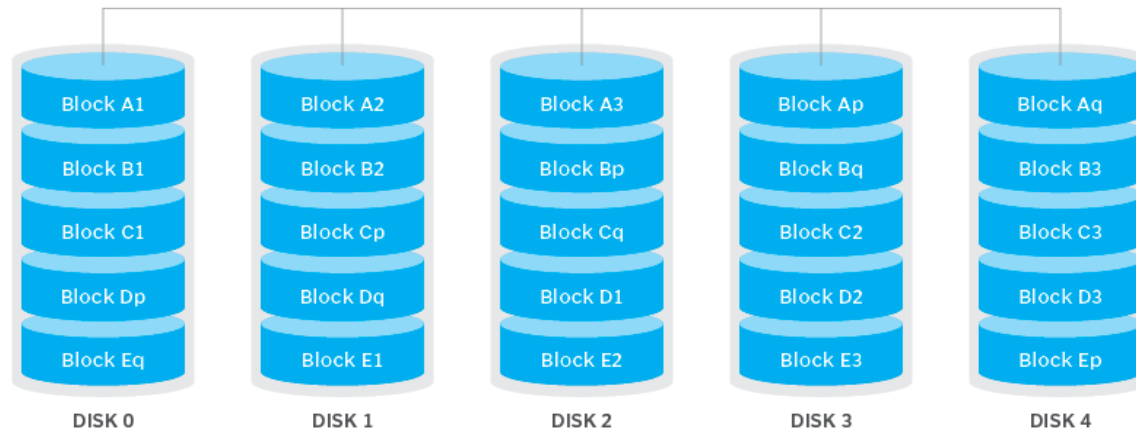


©2018 TECHTARI

RAID 5

RAID 5 stripes parity data across drives, providing a performance boost and allowing one failed drive in the array to rebuild while operations continue..

RAID 6



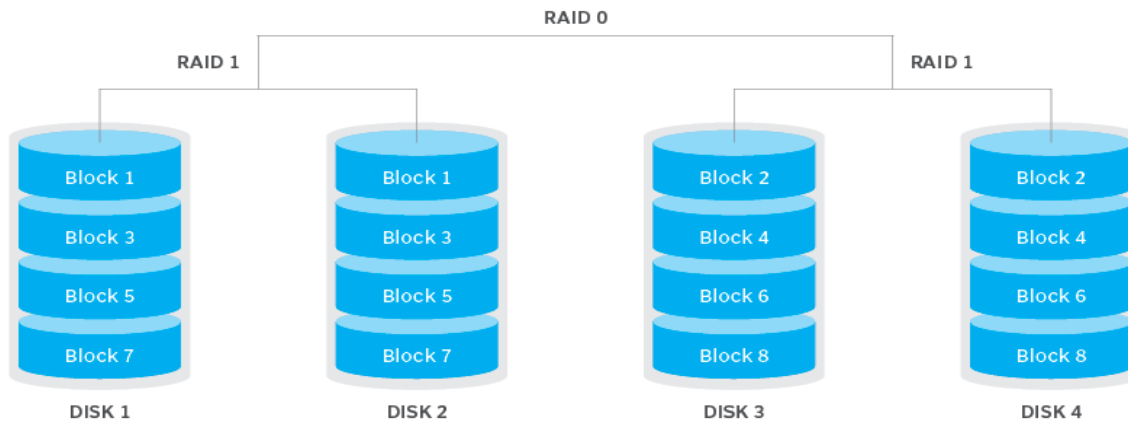
©2018 TECHTARGET. ALL RIGHTS RESERVED.

RAID 6

RAID 6 is similar to RAID 5, but it adds technology to withstand the simultaneous failure of two drives.

RAID 10 (RAID 1+0)

Stripe + mirror



©2018 TECHTARGET. ALL RIGHTS RESERVED

RAID 10

RAID 10 is a combination of RAID 0 and RAID 1, providing improved performance and data protection via redundancy.

LEVEL 6



Independent Data Disks with Double Parity

LEVEL 5



Block Interleaved Distributed Parity

LEVEL 4



Dedicated Parity Drive

LEVEL 3



Bit-Interleaved Parity

LEVEL 2



Error-Correcting Coding

LEVEL 1



Mirroring and Duplexing

LEVEL 0



Striped Disk Array without Fault Tolerance

RAID SUMMARY

RAID Levels Explained